카메라 - 라즈베리 - 아두이노 구현 코드 (rasberry 코드)

◦ 헤더

```
import cv2
import mediapipe as mp        // Open cv 와 mediapipe 라이브러리 이용

import serial
import time                   // 아두이노와 Serial 통신 을 위한 도구 묶러리.
                              아두이노가 처음 리셋되고 통신준비를 마칠때까지
                              기다리는 데 필요한 도구
```

◦ 아두이노 연결

```
try:
    ser = serial.Serial('/dev/ttyUSB0', 9600, timeout=1)   ← 연결시도
    time.sleep(2)   ← 2초 기다린후.
    print("아두이노 연결 성공!")

except Exception as e:   ← 실패시, 실행
    print(f"아두이노 연결 실패: {e}")   // try가 안되면 except으로 간다.
    exit()
```

◦ Media_pipe 설정

```
mp_pose = mp.solutions.pose        // mp_pose 변수에 mediapipe 라이브러리 안에 있는 pose모듈 저장.
pose = mp_pose.Pose(model_complexity=1,
                    min_detection_confidence=0.4,
                    min_tracking_confidence=0.5)   // Pose 모듈을 구체화
mp_draw = mp.solutions.drawing_utils   // mediapipe 라이브러리안에서 drawing_utils 모듈을 변수 mp_draw 에 저장
```

(= 모델 복잡도, 모델 신뢰도, 추적 신뢰도)

pose 모듈의 결과를 시각화 하기 위한 도구.

◦ 카메라 설정

```
cap = cv2.VideoCapture(0, cv2.CAP_V4L2)   // 클래스(≒함수)(=객체) 의 결과를 cap변수에 저장
STOP_THRESHOLD = 0.45   // 모터 정지를 위한 기준값.
```

0 과 cv2.CAP_... 를인자로 갖는다.

0: 첫번째 카메라 사용

cv2.CAP_...: 카메라 연결안정화

↓

카메라 장치 번호, 카메라 열렸는지 여부, 해상도 정보, ... 정보 저장

morning glory

<Tip> 클래스 와 함수

함수: (python)

```
def add_one(x):
    x = x+1
    return x
```

```
a = 0
a = add_one(a)   ... 1 (결과)
a = add_one(a)   -- 2
```

결과를 다시 인자로 넣는다.

클래스: (python)

```
class Counter:
    def __init__(self):
        self.value = 0
    def add_one(self):
        self.value += 1
        return self.value
```

클래스 안의 함수 (메서드)

클래스

```
C = Counter()
C.add_one()   -- 1
C.add_one()   --- 2
```

class를 선언하고 class만 반복해서 불러와도 결과가 누적되어 저장된다.

* 클래스는 C++에 만 존재 (C에는 X)

○ 카메라 알고리즘 실행

print("프레임 이탈 기반 낙상 감지 시스템 시작, 'q' 누르면 종료.")

```
try:                                   // try: ~ , finally: ~ 구조 시작
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break              // ret이 false 이면 while문 종료
        frame = cv2.resize(frame, (640, 480))   // 프레임 크기를 640×480으로 고정.
        img_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = pose.process(img_rgb)   // Mediapipe pose 모듈 실행, 결과저장.
        status_text = "Searching..."
        color = (255, 255, 255)           // 상태표시.
        if results.pose_landmarks:
            mp_draw.draw_landmarks(frame, results.pose_landmarks, mp_pose.POSE_CONNECTIONS)
            lm = results.pose_landmarks.landmark
```

카메라가 정상적으로 열린동안 반복.

cap.read() = 카메라에서 프레임을 읽는다 → ret에 성공여부 저장, frame에 읽은 영상 프레임 저장(배열로)

해상도조정 함수

open cv라이브러리

pose 모듈

모듈 실행 함수

results 안의 Data 中 관절좌표 Data가 존재하면,

frame 위에 results.pose_landmarks (=관절점)을 가지고 POSE_CONN (=연결규칙)으로 관절을 연결하여 시각적으로 표현한다.

// results 안에 있는 관절좌표 Data (pose_landmarks) 안에 있는 실제좌표값 (landmark)를 lm에 저장

Note. try ~ finally ~

try ~ 수행후 break / error / ... 이 발생하면, 무조건 finally를 수행하고 마친다. 에러가 나도, finally는 수행하고 끝난다.

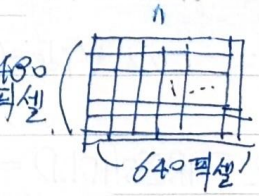
// open CV의 BGR 이미지를 Mediapipe의 RGB 형식으로 변환

Note. 프레임 과 프레임 크기

프레임 = 영상의 한 장면 = 사진 = 배열

프레임 크기 = 픽셀 갯수

(480, 640, 3)



& 색상 채널수 = 3개.

픽셀 하나속 정보 = [B, G, R]

(색상채널 3개인경우) = [255, 0, 0] = Blue, [0, 255, 0] = Green, [0, 0, 255] = Red

morning glory

ote. 속성 과 함수

속성: results.pose_landmarks.landmark = 저장된 값을 읽어온다

└ results 안에 존재하는 pose_landmarks 데이터 가지고(불러와서) 그 안에 존재하는 landmark 데이터 불러오기

함수: pose.process(img_rgb) = 특정 명령을 실행한다

└ img_rgb 데이터를 대상으로 pose 모듈을 process (=실행) 한다

(X.y(z)): X = object = 주체
y = method = 함수
z = 인자 = 대상
⇒ z를 가지고 X가 y를 실행한다

// lm[0], lm[1], lm[2] ··· lm[32] = 33개의 관절 정보 리스트

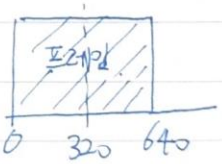
lm[0].X = 0번 관절의 X좌표(가로)
lm[0].y = 0번 관절의 y 좌표(세로)
lm[0].z = 0번 관절의 z 좌표(거리)
lm[0].visibility = 0번 관절이 확실히 보이는 정도

(while)
(if)

// 좌표추출

CX = int((lm[11].X + lm[12].X) * 640/2) — 어깨의 중심 좌표 CX에 저장

// lm[ ]는 좌표의 비율이다
eg) 0.5 = X좌표 320
0.1 = X좌표 64



error = CX - 320 — 어깨중심좌표와 프레임 중심 사이 픽셀 거리차

Shoulder_width = abs(lm[11].X - lm[12].X) — 어깨너비 계산 (for 거리계산)

// 아두이노 DC 모터 제어

if Shoulder_width > STOP_THRESHOLD: // 정지

Ser.write("999\n".encode())

Status_text = "TOO CLOSE - STOPPED"

Color = (0, 165, 255)

화면에 문구 표기

//아두이노에 모터 정지 신호 전달 (아주큰 숫자 999 전송)

else: // 정상추적

Ser.write(f"{error}\n".encode()) — 아두이노에 error (=숫자)를 보낸다

Status_text = f"Tracking - Error: {error}"

Color = (0, 255, 0)

화면에 문구 표기

frame 위에 (CX, 240)의 좌표위치에 반지름이 10 이고 두께가 -1인 원을 그린다

CV2.circle(frame, (CX, 240), 10, color, -1) // 어깨 중심점 시각화

└ opencv라이브러리 안에 └ circle 이라는 시각화 도구 함수

└ 인자들

morning glory

```
else:                    // results 안에 Data 中 관절 좌표 Data가 존재하지 않으면 = 사람이 프레임 안에 없으면
    ser.write("999\n".encode())   // 정지 신호 아두이노에 전송
    status_text = "FALL DETECTED (Missing) - STOP"
    color(0, 0, 255)
```

화면에 문구 표시

화면에 문구 시각화 을 위한 기본 설정.

(if문 종료)

openCV 라이브러리 안에 putText 함수 이용

프레임 위에 / 메시지를 표시 / 화면 왼쪽 상단에 / 글꼴 / 글자 크기/색상/두께

```
cv2.putText(frame, status_text, (10, 50), cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
cv2.imshow("Atov Tracking System", frame)
```

— "ATOV ~" 라는 이름의 윈도우 창에 최종 영상 출력

```
if cv2.waitKey(1) & 0xFF == ord('q'):
    break
```

break — while 문 종료

while문 종료 조건

사용자가 q를 누르면

(while문 종료)

```
finally:                 // 마지막에 항상 실행.
    if 'ser' in locals() and ser.is_open:   // 연결이 되어 있는지 확인
        ser.write("999\n".encode())         // 로봇정지 신호전송
        ser.close()
cap.release()            // 카메라 종료
cv2.destroyAllWindows()  // 윈도우 창("ATOV Tracking System") 닫기.
```